

Project Report: Highway Search Algorithms

CS (NW) 36210 002 – Introduction To Artificial Intelligence, Fall 2026
Purdue University Northwest
10/08/2026

Class ID (Ascend sequency)	Contributor Name (Last Name, First Name)	Email	Detailed Contributions
7	Singh, Manjot	sing1836@pnw.edu	BFS and DFS
8	Amereh, Motasem	mamereh@pnw.edu	A* and System Integration
9	Bhatia, Parv Alphonso	bhati127@pnw.edu	Visualization and Experimental Results
11	Devi, Tamanna	tdevi@pnw.edu	Google Maps Data and Heuristic Preparation
12	Zangrilli, Thomas	tzangril@pnw.edu	Uniform-Cost Search and Testing

Table of Contents

Abstract.....	3
I. Introduction.....	3
II. Collecting the Data.....	5
1. Choosing the cities.....	5
2. Driving distances $g(n)$	6
3. Straight-line distances $h(n)$	7
III. Search Algorithms.....	8
1. Problem formulation.....	8
2. Breadth-First Search.....	9
3. Depth-First Search.....	10
4. Uniform Cost Search.....	10
5. A* Search.....	10
IV. Program Implementation.....	11
1. Program structure.....	11
2. Running the program.....	12
3. Showing the travel map.....	12
V. Results.....	13
1. User-selected route: Chicago to Atlanta.....	13
2. Five random tests.....	15
VI. Discussion.....	18
1. Communication problems.....	18
2. Different ideas about the project.....	19
3. Change of group president.....	19
VII. Conclusion.....	19
Acknowledge.....	20
Appendix.....	20
1. Source code: highway_search.py.....	20
2. Source code: city_data.py.....	23

Table of Tables

Table 1 – Cities used in the highway network.....	5
---	---

Table 2 – Driving distances between connected cities, $g(n)$	6
Table 3 – Straight-line distances between all city pairs in miles, $h(n)$	8
Table 4 – Comparison of the four search algorithms.....	11
Table 5 – Results for Chicago to Atlanta.....	13
Table 6 – Results of the five random tests (seed 42).....	15

Table of Figures

Figure 1 - Requirements of the Highway Search project.....	4
Figure 2 - The 11-city highway network with driving distances in miles.....	7
Figure 3 - The search loop shared by BFS, DFS, UCS and A*	9
Figure 4 - Program output for Chicago to Atlanta.....	13
Figure 5 - Travel map window for Chicago to Atlanta.....	14
Figure 6 - Program output for the five random tests.....	15
Figure 7 - Random test 1: Nashville to Chicago.....	16
Figure 8 - Random test 2: Saint Paul to Columbus.....	16
Figure 9 - Random test 3: Indianapolis to Nashville.....	17
Figure 10 - Random test 4: Springfield to Chicago.....	17
Figure 11 - Random test 5: Nashville to Columbia.....	18

ABSTRACT

Search is one of the first problem-solving tools taught in artificial intelligence. An agent that has to get from one state to another, such as a driver going from one city to another, can treat the problem as a graph and search that graph for a path. In this project our team built a highway map of 11 cities in 9 states and the District of Columbia, collected the real driving distance of every highway connection from Google Maps as the path cost $g(n)$, and measured the straight-line (airplane) distance between every pair of cities as the heuristic $h(n)$.

We implemented four search algorithms in Python: Breadth-First Search (BFS), Depth-First Search (DFS), Uniform Cost Search (UCS), and A* search. The program accepts any start and target city, or picks five random city pairs, runs all four algorithms, and shows each result on a travel map with the route, the driving distance, and the order in which cities were expanded. The straight-line distance is also converted into an airplane travel time at 250 miles per hour.

In our runs, one route chosen by the user and five random tests, UCS and A* always returned the shortest driving route, while BFS and DFS sometimes returned longer ones. A* found the same shortest routes as UCS while expanding fewer cities in five of the six runs, showing how a heuristic reduces the work of the search without giving up the optimal answer. This report describes the data collection, the four algorithms, the program, and the results.

I. INTRODUCTION

Many problems in artificial intelligence can be described as finding a sequence of actions that leads from a starting state to a goal state. Route finding is the classic example: the states are cities, the actions are driving along a highway, and the cost of an action is the number of miles driven. Because the problem is easy to picture on a map, it is a good way to see how different search strategies behave and why some of them find better routes or do less work than others.

Search algorithms fall into two groups. *Uninformed* algorithms such as BFS, DFS, and UCS only know the graph itself; they cannot tell whether a city is getting closer to the target. *Informed* algorithms such as A* also use a heuristic, an estimate of the remaining cost to the goal. In this project the actual cost so far is the driving distance $g(n)$, the heuristic $h(n)$ is the straight-line distance from city n to the target, and A* ranks cities by $f(n) = g(n) + h(n)$. The straight-line distance is a natural heuristic for driving, because no road between two cities can be shorter than the straight line between them.

The project asked each team to (1) create a highway network of 10 to 15 cities with at least one city in each state used, (2) collect the driving distances and straight-line distances from Google Maps, (3) write BFS, DFS, UCS, and A* in one programming language, and (4) document the work in a report, a presentation, and a recorded video. The program must support two kinds of runs: a route between any two cities the user chooses, and five randomly chosen start and target cities, each showing a travel map and the distances. These requirements are summarized in Figure 1.

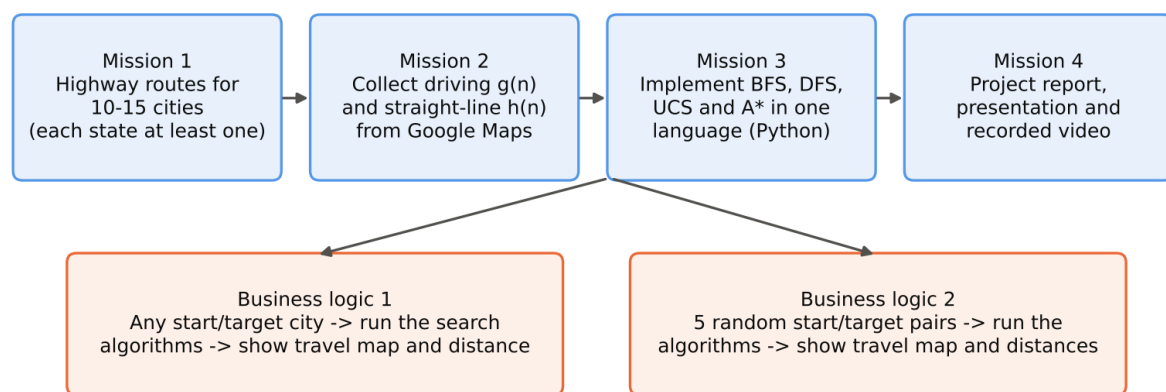


Figure 1 - Requirements of the Highway Search project

The rest of this report follows the order of the work. Section II describes the cities and the data we collected. Section III explains the four algorithms. Section IV describes the Python program and how it draws the travel map. Section V presents the results for a user-selected route

and the five random tests. Section VI discusses the challenges our team faced, and Section VII summarizes the project.

II. COLLECTING THE DATA

All distances in this project come from Google Maps. Two different kinds of distance were needed: the driving distance along highways between neighboring cities, which is the real cost of travel $g(n)$, and the straight-line distance between every pair of cities, which is the heuristic $h(n)$. Both are stored in the file `city_data.py` (Appendix 2) so that the search program and the data are kept separate.

1. Choosing the cities

Our highway network has 11 cities, listed in Table 1. They are spread over 9 states and the District of Columbia; Chicago and Springfield are both in Illinois, which is allowed because the requirement is that every state used has at least one city. Each city is given a three-letter abbreviation, which is used in the larger tables later in this report.

Table 1 – Cities used in the highway network

Abbr.	City	State	Highway connections
STP	Saint Paul	Minnesota	2
CHI	Chicago	Illinois	4
SPI	Springfield	Illinois	4
IND	Indianapolis	Indiana	4
CMH	Columbus	Ohio	4
PHL	Philadelphia	Pennsylvania	3
NYC	New York	New York	1
DCA	Washington DC	District of Columbia	3
CAE	Columbia	South Carolina	3
ATL	Atlanta	Georgia	2
BNA	Nashville	Tennessee	4

2. Driving distances $g(n)$

The driving distance of each highway connection was collected from Google Maps. Our network has 17 highway connections, listed in Table 2. Every connection can be driven in both directions with the same distance, so the program stores the network as an undirected graph.

Table 2 – Driving distances between connected cities, $g(n)$

From	To	Driving distance (miles)
Saint Paul	Chicago	398
Saint Paul	Springfield	515
Springfield	Chicago	201
Springfield	Nashville	374
Springfield	Indianapolis	211
Chicago	Indianapolis	183
Chicago	Columbus	325
Indianapolis	Nashville	288
Indianapolis	Columbus	174
Nashville	Columbia	462
Nashville	Atlanta	248
Columbus	Philadelphia	469
Columbus	Washington DC	398
Atlanta	Columbia	214
Columbia	Washington DC	478
Washington DC	Philadelphia	136
Philadelphia	New York	95

Figure 2 shows the resulting highway network. The cities are placed by their latitude and longitude so the map looks like the real geography, and each road is labeled with its driving distance. This drawing is produced by the same program function that draws the search results in Section V.

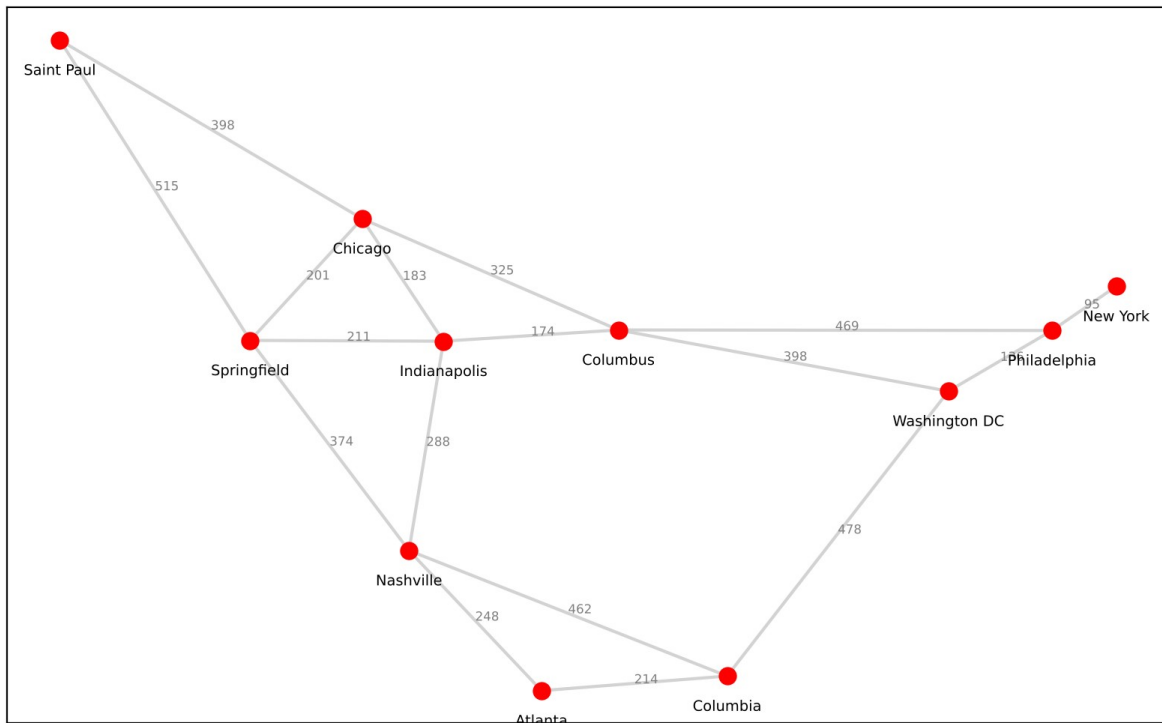


Figure 2 - The 11-city highway network with driving distances in miles

3. Straight-line distances $h(n)$

The heuristic needs the straight-line distance from every city to every possible target, so all 55 city pairs had to be measured, not only the 17 connected pairs. These distances were measured through Google Maps and are given in Table 3 using the abbreviations from Table 1. The table is symmetric because the distance from A to B is the same as from B to A, and the diagonal is zero because a city is zero miles from itself.

Table 3 – Straight-line distances between all city pairs in miles, $h(n)$

	STP	CHI	SPI	IND	CMH	PHL	NYC	DCA	CAE	ATL	BNA
STP	0	346.44	397.21	505.89	619.46	974.7	1007.55	922.29	991.33	901.326	691.88
CHI	346.44	0	179.51	165.34	291.66	663.84	711.148	594.225	652.39	588.091	397.628
SPI	397.21	179.51	0	186	352.246	767.487	824.93	675.288	621.64	508.823	296.54
IND	505.89	165.34	186	0	169.24	583.29	643.35	492.189	488.603	426.476	251.5
CMH	619.46	291.66	352.246	169.24	0	414.98	475.44	326.6	426.119	435.507	333.711
PHL	974.7	663.84	767.487	583.29	414.98	0	80.62	123.41	523.569	663.913	683.559
NYC	1007.55	711.148	824.93	643.35	475.44	80.62	0	202.26	523.73	745.027	682.955
DCA	922.29	594.225	675.288	492.189	326.6	123.41	202.26	0	405.32	542.443	566.509
CAE	991.33	652.39	621.64	488.603	426.119	523.569	523.73	405.32	0	193.14	357.54
ATL	901.326	588.091	508.823	426.476	435.507	663.913	745.027	542.443	193.14	0	214.81
BNA	691.88	397.628	296.54	251.5	333.711	683.559	682.955	566.509	357.54	214.81	0

The project also asks for the travel time of an airplane flying the straight-line distance at 250 miles per hour. The program computes this as $h / 250$. For example, Chicago to Atlanta is 588.09 miles in a straight line, which is 2.35 hours by airplane.

Straight-line distance works as a heuristic for A* because no road between two cities can be shorter than the straight line between them. This means $h(n)$ never overestimates the real remaining driving distance, a property called *admissibility*, which is what A* needs to be sure of returning the shortest route.

III. SEARCH ALGORITHMS

All four algorithms solve the same problem and return the same two things: the final path from the start city to the target city, and the list of cities in the order they were expanded. A city is *expanded* when the algorithm takes it out of the frontier and looks at its neighbors. The number of expanded cities is the measure of work we use to compare the algorithms.

1. Problem formulation

Each state is a city. The actions available in a city are the highway connections leaving it, and the cost of an action is the driving distance of that road from Table 2. The goal test checks

whether the current city is the target. To keep the code simple, every algorithm stores whole paths in its frontier instead of single cities, so when the target is reached its path is already built. The algorithms share the same loop (Figure 3) and differ only in which path they take out of the frontier next.

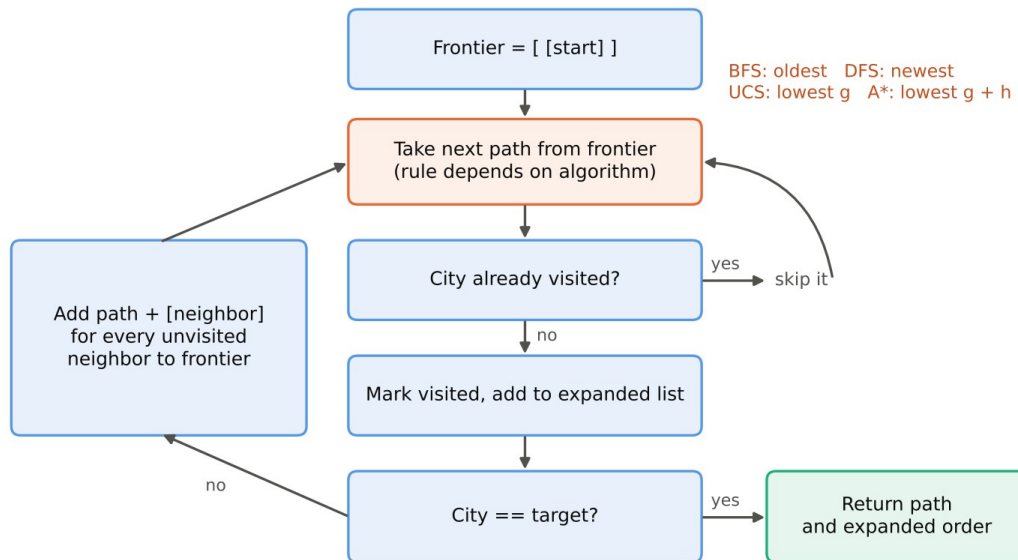


Figure 3 - The search loop shared by BFS, DFS, UCS and A*

Each algorithm keeps a set of visited cities so that no city is expanded twice. This makes the search a graph search rather than a tree search, which matters because our highway network has cycles, for example Chicago, Indianapolis, and Springfield are all connected to each other.

2. Breadth-First Search

BFS uses a first-in, first-out queue, so it always expands the oldest path in the frontier. As a result it explores all cities one road away from the start, then all cities two roads away, and so on. BFS marks a city as visited when the city is added to the queue rather than when it is expanded, which prevents the same city from being added twice. BFS is complete and always finds the route

with the fewest roads, but it ignores the length of the roads, so the route with the fewest roads is not always the route with the fewest miles.

3. Depth-First Search

DFS uses a last-in, first-out stack, so it always expands the newest path. It follows one road as far as it can before backing up to try another. The neighbors of a city are pushed onto the stack in reverse order so that they are explored in the order they are listed in the graph. Because the frontier can hold more than one path to the same city, DFS checks when a path is taken out whether its last city was already visited and skips it if so. DFS is complete on a finite graph with a visited set, but it is not optimal: the first route it finds can be very long.

4. Uniform Cost Search

UCS uses a priority queue ordered by $g(n)$, the total driving distance of the path so far. We use Python's `heapq` module, which always removes the smallest item. Each item in the queue is a pair (g, path) . Because the cheapest path is always expanded first, the first time a city is taken out of the queue its path is the shortest one, so any later copies of that city are skipped. As a result, UCS always finds the shortest driving route.

5. A* Search

A* works exactly like UCS except that the priority is $f(n) = g(n) + h(n)$: the miles already driven plus the straight-line miles still left to the target. Cities that lead away from the target get a large h and are pushed back in the queue, so A* spends less time exploring in the wrong direction. Because our heuristic is admissible, A* still returns the shortest route. The core of our A* code is shown below; the full program is in Appendix 1.

```

frontier = [(heuristic(start, goal), 0, [start])]
visited = set()
expanded = []

while frontier:
    f, g, path = heapq.heappop(frontier)
    city = path[-1]
    if city in visited:
        continue
    visited.add(city)
    expanded.append(city)
    if city == goal:
        return path, expanded

    for neighbor, miles in GRAPH[city].items():
        if neighbor not in visited:
            new_g = g + miles
            new_f = new_g + heuristic(neighbor, goal)
            heapq.heappush(frontier, (new_f, new_g, path + [neighbor]))

```

Table 4 summarizes the four algorithms.

Table 4 – Comparison of the four search algorithms

Algorithm	Frontier	Takes out next	Uses h(n)	Shortest route?
BFS	Queue (FIFO)	Oldest path	No	Fewest roads only
DFS	Stack (LIFO)	Newest path	No	No
UCS	Priority queue	Lowest g(n)	No	Yes
A*	Priority queue	Lowest g(n) + h(n)	Yes	Yes (h admissible)

IV. PROGRAM IMPLEMENTATION

1. Program structure

The program is written in Python 3 and is split into two files. `city_data.py` holds the list of cities, their coordinates (used only for drawing), the driving distances, the straight-line distances, and a `heuristic(city, goal)` function that looks up h for any pair. It also builds `GRAPH`, a dictionary in which `GRAPH[city]` maps each neighbor to its driving distance. `highway_search.py` contains the four search functions, a `path_distance` helper that adds up the miles of a path, the functions that print results and draw maps, and a text menu. The only library outside Python itself is Matplotlib, which draws the maps.

Each search function has the same signature, `algorithm(start, goal)`, and returns `(final_path, expanded_order)`. The four functions are stored in a dictionary, `ALGORITHMS`, so

the rest of the program can run all of them in a simple loop without treating any algorithm as a special case.

2. Running the program

The program is started from a terminal with `python highway_search.py` and shows a menu with three options. Option 1 lists the cities by number and asks for a start city and a target city; the program rejects a target that is the same as the start. Option 2 runs five random tests. Option 3 quits.

After the cities are chosen, the program prints the straight-line distance h between the start and target, the airplane time at 250 miles per hour, and one line per algorithm with the driving distance, the number of expanded cities, and the route. The program output for every run in this report is shown with its results in Section V.

3. Showing the travel map

After printing the results, the program opens one window with a two-by-two grid of maps, one for each algorithm, so the four results can be compared side by side. Each map draws every highway in gray with its driving distance and the final route in blue. Cities that the algorithm expanded are drawn in orange and numbered in the order they were expanded; cities that were never expanded stay red. This shows both what each algorithm found and how much work it did to find it. For the random tests, the program shows the map for each test in turn; closing one window opens the next.

V. RESULTS

1. User-selected route: Chicago to Atlanta

For the first business-logic requirement we ran the program with option 1, choosing city 2 (Chicago) as the start and city 10 (Atlanta) as the target. The straight-line distance is 588.09 miles, or 2.35 hours by airplane. Figure 4 shows the program output, Table 5 lists each algorithm's result, and Figure 5 shows the travel map window.

```

=== Highway Search ===
1. Pick a start and target city
2. Run 5 random tests
3. Quit
Choose 1, 2, or 3: 1
  1. Saint Paul
  2. Chicago
  3. Springfield
  4. Indianapolis
  5. Columbus
  6. Philadelphia
  7. New York
  8. Washington DC
  9. Columbia
 10. Atlanta
 11. Nashville
Start city number: 2
  1. Saint Paul
  2. Chicago
  3. Springfield
  4. Indianapolis
  5. Columbus
  6. Philadelphia
  7. New York
  8. Washington DC
  9. Columbia
 10. Atlanta
 11. Nashville
Target city number: 10

Chicago -> Atlanta
Straight-line h = 588.1 miles | Airplane @ 250 mph = 2.35 hours
BFS   823 miles | 10 expanded | Chicago -> Springfield -> Nashville -> Atlanta
DFS  3046 miles | 10 expanded | Chicago -> Saint Paul -> Springfield -> Nashville -> Indianapolis -> Columbus -> Philadelphia -> Washington DC -> Columbia ->
Atlanta
UCS   719 miles | 7 expanded | Chicago -> Indianapolis -> Nashville -> Atlanta
A*   719 miles | 5 expanded | Chicago -> Indianapolis -> Nashville -> Atlanta

```

Figure 4 - Program output for Chicago to Atlanta

Table 5 – Results for Chicago to Atlanta

Algorithm	Route	Miles	Expanded	Expansion order
BFS	CHI → SPI → BNA → ATL	823	10	CHI, STP, SPI, IND, CMH, BNA, PHL, DCA, CAE, ATL
DFS	CHI → STP → SPI → BNA → IND → CMH → PHL → DCA → CAE → ATL	3,046	10	CHI, STP, SPI, BNA, IND, CMH, PHL, DCA, CAE, ATL
UCS	CHI → IND → BNA → ATL	719	7	CHI, IND, SPI, CMH, STP, BNA, ATL
A*	CHI → IND → BNA → ATL	719	5	CHI, IND, BNA, SPI, ATL

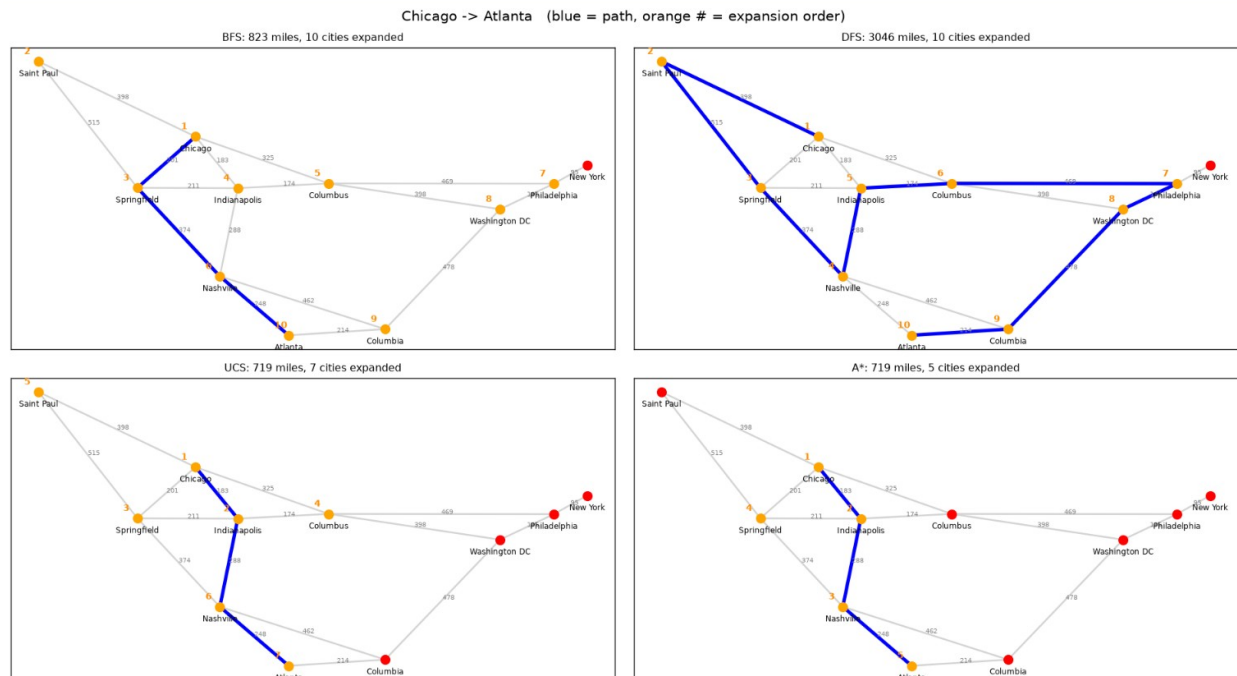


Figure 5 - Travel map window for Chicago to Atlanta

UCS and A* both found the shortest route, 719 miles through Indianapolis and Nashville. BFS returned an 823-mile route through Springfield and Nashville instead. Both routes use three roads, so to BFS they are equally good; it took the Springfield route only because Springfield is listed before Indianapolis among Chicago's neighbors and was expanded first. That choice cost 104 extra miles. DFS went the long way around through Saint Paul, the East Coast, and Columbia, a 3,046-mile route that is more than four times longer than necessary.

The difference in work is also clear on the map. BFS and DFS expanded 10 of the 11 cities. UCS expanded 7, including Saint Paul and Columbia, which are in the wrong direction, because it only looks at the miles already driven. A* expanded only 5 cities; the heuristic told it that Saint Paul, Columbia, Columbia, and the East Coast cities were far from Atlanta, so it never needed to expand them.

2. Five random tests

For the second business-logic requirement we ran the program with option 2, which picks five random start and target pairs. We set the random seed to 42 in the program so that the same five pairs are produced every time, which lets anyone repeat our results and lets the report and the video show the same tests. Figure 6 shows the program output, Table 6 lists the results of all five tests, and Figures 7 to 11 show the travel map window for each test.

```

=== Highway Search ===
1. Pick a start and target city
2. Run 5 random tests
3. Quit
Choose 1, 2, or 3: 2

--- Random test 1 ---
Nashville -> Chicago
Straight-line h = 397.6 miles | Airplane @ 250 mph = 1.59 hours
BFS  575 miles | 7 expanded | Nashville -> Springfield -> Chicago
DFS  1287 miles | 4 expanded | Nashville -> Springfield -> Saint Paul -> Chicago
UCS  471 miles | 7 expanded | Nashville -> Indianapolis -> Chicago
A*   471 miles | 3 expanded | Nashville -> Indianapolis -> Chicago

--- Random test 2 ---
Saint Paul -> Columbus
Straight-line h = 619.5 miles | Airplane @ 250 mph = 2.48 hours
BFS  723 miles | 5 expanded | Saint Paul -> Chicago -> Columbus
DFS  1435 miles | 6 expanded | Saint Paul -> Chicago -> Springfield -> Nashville -> Indianapolis -> Columbus
UCS  723 miles | 5 expanded | Saint Paul -> Chicago -> Columbus
A*   723 miles | 3 expanded | Saint Paul -> Chicago -> Columbus

--- Random test 3 ---
Indianapolis -> Nashville
Straight-line h = 251.5 miles | Airplane @ 250 mph = 1.01 hours
BFS  288 miles | 4 expanded | Indianapolis -> Nashville
DFS  2994 miles | 9 expanded | Indianapolis -> Springfield -> Saint Paul -> Chicago -> Columbus -> Philadelphia -> Washington DC -> Columbia -> Nashville
UCS  288 miles | 5 expanded | Indianapolis -> Nashville
A*   288 miles | 2 expanded | Indianapolis -> Nashville

--- Random test 4 ---
Springfield -> Chicago
Straight-line h = 179.5 miles | Airplane @ 250 mph = 0.72 hours
BFS  201 miles | 3 expanded | Springfield -> Chicago
DFS  913 miles | 3 expanded | Springfield -> Saint Paul -> Chicago
UCS  201 miles | 2 expanded | Springfield -> Chicago
A*   201 miles | 2 expanded | Springfield -> Chicago

--- Random test 5 ---
Nashville -> Columbia
Straight-line h = 357.5 miles | Airplane @ 250 mph = 1.43 hours
BFS  462 miles | 4 expanded | Nashville -> Columbia
DFS  2727 miles | 9 expanded | Nashville -> Springfield -> Saint Paul -> Chicago -> Indianapolis -> Columbus -> Philadelphia -> Washington DC -> Columbia
UCS  462 miles | 5 expanded | Nashville -> Atlanta -> Columbia
A*   462 miles | 3 expanded | Nashville -> Atlanta -> Columbia

```

Figure 6 - Program output for the five random tests

Table 6 – Results of the five random tests (seed 42)

Test	Start → target	Algorithm	Miles	Expanded	Route
1	BNA → CHI h = 397.6 mi 1.59 h by air	BFS	575	7	BNA → SPI → CHI
		DFS	1,287	4	BNA → SPI → STP → CHI
		UCS	471	7	BNA → IND → CHI
		A*	471	3	BNA → IND → CHI

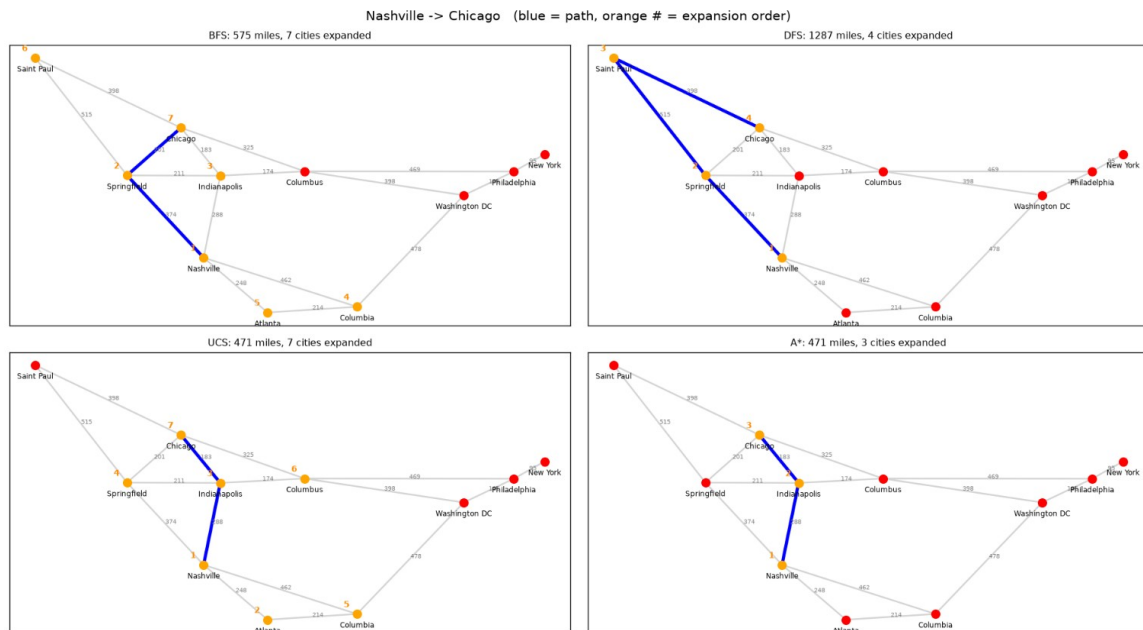


Figure 7 - Random test 1: Nashville to Chicago

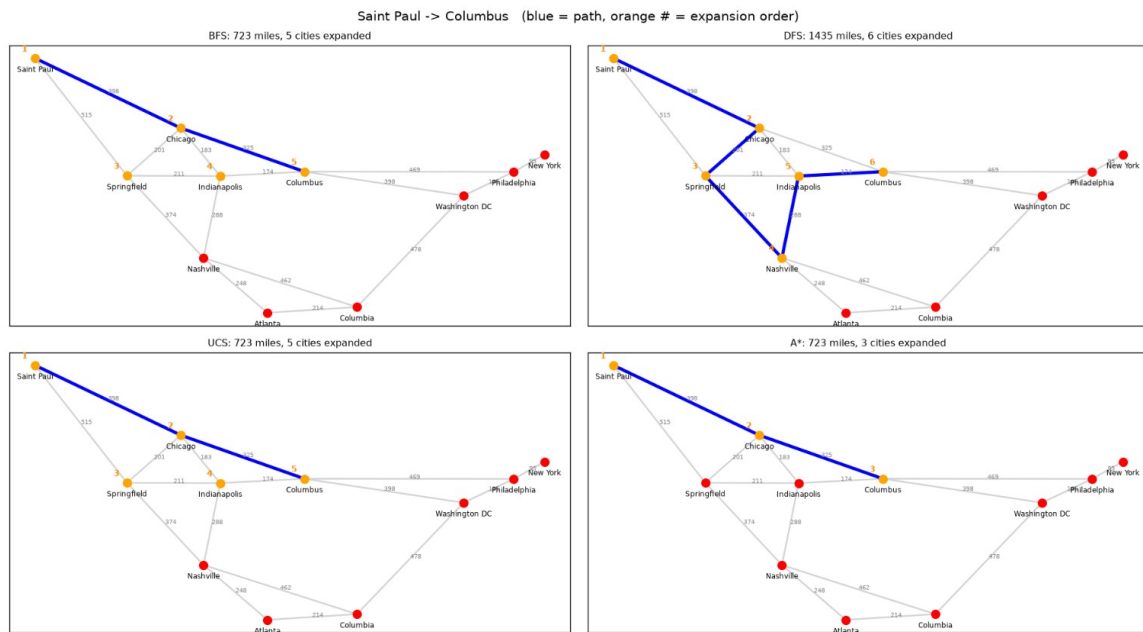


Figure 8 - Random test 2: Saint Paul to Columbus

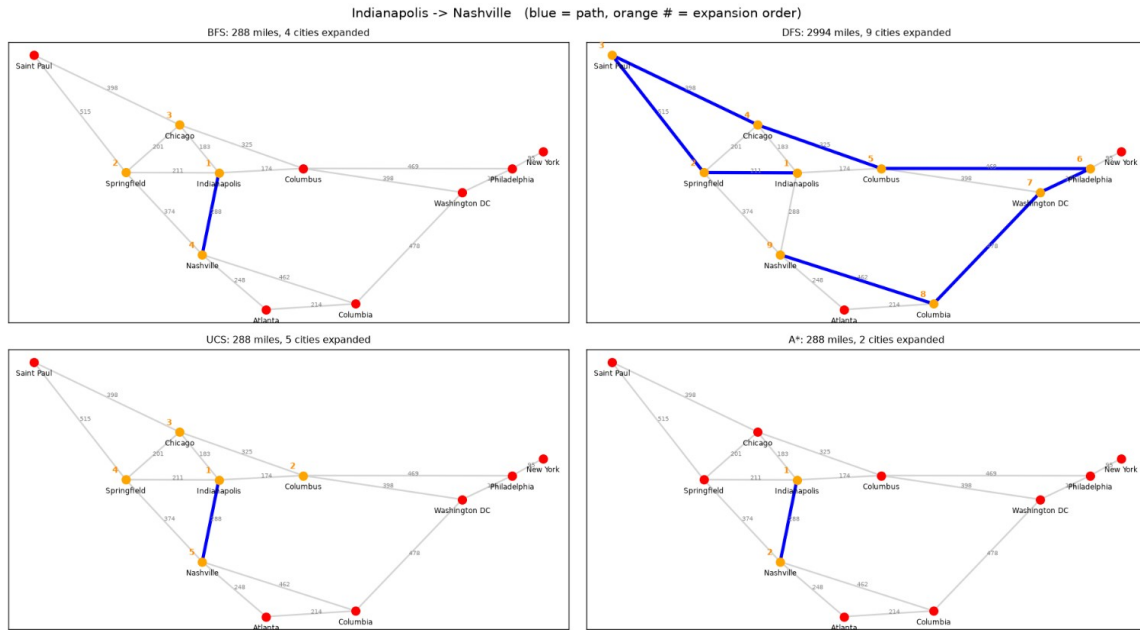


Figure 9 - Random test 3: Indianapolis to Nashville

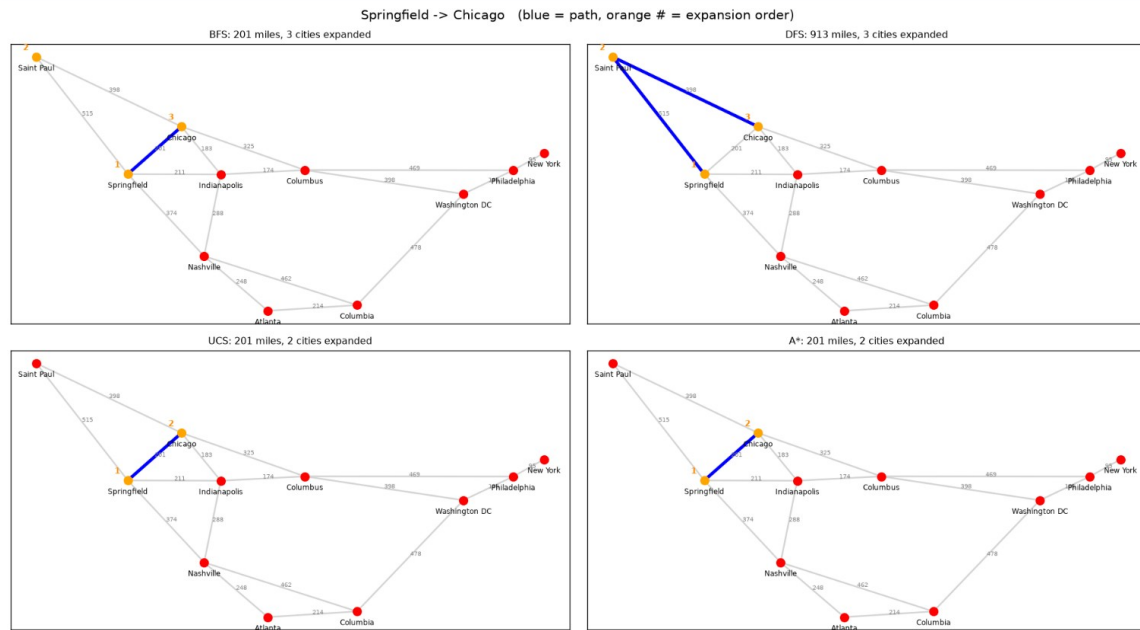
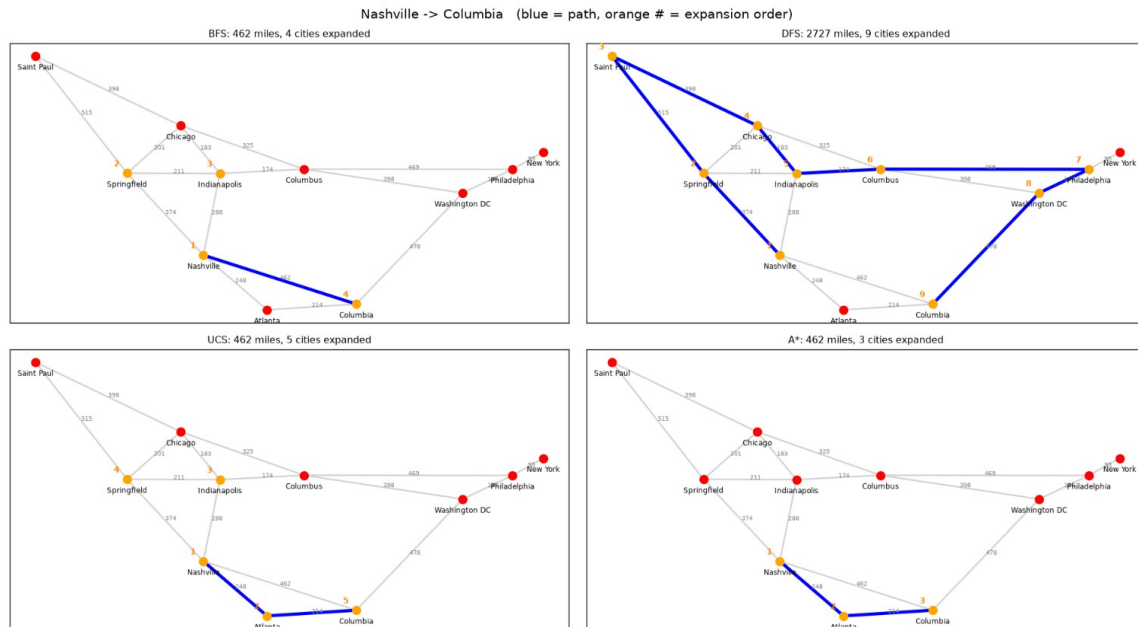


Figure 10 - Random test 4: Springfield to Chicago



In all five tests UCS and A* returned the shortest route, and A* expanded fewer cities than UCS in 4 of the five tests. The tests also show both weaknesses of the uninformed algorithms. In test 1 (Nashville to Chicago), BFS chose the two-road route through Springfield (575 miles) even though the route through Indianapolis is also two roads and only 471 miles; BFS has no way to tell them apart. In test 3 (Indianapolis to Nashville) the two cities are directly connected, yet DFS wandered through 7 other cities and drove 2,994 miles instead of 288.

VI. DISCUSSION

Along with the technical work, our team faced some challenges in working together. These included (1) communication problems, (2) different ideas about what the project should include and how to do it, and (3) a change of group president during the project. Each is discussed below.

1. Communication problems

Keeping communication clear between all five team members was a challenge at times during the project. We worked through it and completed the project together.

2. Different ideas about the project

Team members had different ideas about what the project should include and how it should be done. The team discussed these ideas and agreed on one approach for the program and the report.

3. Change of group president

Our group president changed during the project, so the role of coordinating the team was handed over to a new member partway through the work.

VII. CONCLUSION

In this project we built a highway network of 11 cities from Google Maps data and used it to compare four classic search algorithms. Our runs showed the main ideas of the course in a way that is easy to see on a map. BFS finds the route with the fewest roads, which is not always the shortest. DFS finds a route, but often a very long one. UCS always finds the shortest route but also expands cities in the wrong direction. A* finds the same shortest route as UCS while expanding fewer cities, because the straight-line heuristic keeps it pointed toward the target.

The work was divided by algorithm and by stage of the project. Tamanna Devi collected the Google Maps driving distances and straight-line distances and prepared the heuristic data. Manjot Singh implemented BFS and DFS, Thomas Zangrilli implemented Uniform Cost Search and tested the algorithms, and Motasem Amereh implemented A* and integrated the parts into one program. Parv Alphonso Bhatia built the travel map visualization and produced the experimental results.

ACKNOWLEDGE

We would like to thank Professor Wei Dai for the guidance and for a project that let us apply search algorithms to a real problem. We also thank every member of our team for their work and patience in completing the project together.

APPENDIX

1. Source code: highway_search.py

```
# Project: Highway Search Algorithms - Intro to Artificial Intelligence
# Project team: Motasem Amereh, Tamanna Devi, Manjot Singh,
#              Thomas Zangrilli, Parv Alphonso Bhatia
# File: highway_search.py
# Version: 1.0
# Updated: 2026-10-08
# Purpose: Find a route between two of 11 cities with BFS, DFS, UCS, and A*,
#          and show each route and its distance on a map.
# Data: g(n) = Google Maps driving distance, h(n) = Google Maps straight-line
#        distance, airplane time = h / 250 mph (all in city_data.py)
# Requires: Python 3.8+, matplotlib (pip install matplotlib)
# Run: python highway_search.py

import heapq
import random
import matplotlib.pyplot as plt

from city_data import CITIES, CITY_COORDS, GRAPH, AIRPLANE_SPEED_MPH, heuristic

def path_distance(path):
    total = 0
    for i in range(len(path) - 1):
        total += GRAPH[path[i]][path[i + 1]]
    return total

# -----
# Search algorithms
#
# All four keep whole paths in the frontier, so when the goal is
# reached its path is already built. They differ only in WHICH
# path is taken out next:
#   BFS -> oldest path (queue)      DFS -> newest path (stack)
#   UCS -> lowest g (driving miles) A* -> lowest f = g + h
#
# Each returns (final_path, expanded_order).
# -----

def bfs(start, goal):
    queue = [[start]]
    visited = {start}
    expanded = []

    while queue:
        path = queue.pop(0)
        city = path[-1]
        expanded.append(city)
        if city == goal:
            return path, expanded

        for neighbor in GRAPH[city]:
            # Mark visited when added, so no city enters the queue twice.
            if neighbor not in visited:
```

```

        visited.add(neighbor)
        queue.append(path + [neighbor])

    return [], expanded

def dfs(start, goal):
    stack = [[start]]
    visited = set()
    expanded = []

    while stack:
        path = stack.pop()
        city = path[-1]
        # A city can be pushed more than once; only expand it the first time.
        if city in visited:
            continue
        visited.add(city)
        expanded.append(city)
        if city == goal:
            return path, expanded

        # Pushed in reverse so the first listed neighbor is popped first.
        for neighbor in reversed(list(GRAPH[city])):
            if neighbor not in visited:
                stack.append(path + [neighbor])

    return [], expanded

def ucs(start, goal):
    # heapq always pops the smallest tuple, i.e. the path with the lowest g.
    frontier = [(0, [start])]
    visited = set()
    expanded = []

    while frontier:
        g, path = heapq.heappop(frontier)
        city = path[-1]
        # The first time a city is popped is its cheapest route; skip later copies.
        if city in visited:
            continue
        visited.add(city)
        expanded.append(city)
        if city == goal:
            return path, expanded

        for neighbor, miles in GRAPH[city].items():
            if neighbor not in visited:
                heapq.heappush(frontier, (g + miles, path + [neighbor]))

    return [], expanded

def astar(start, goal):
    # Same as UCS, but ordered by  $f = g + h$ . Since  $h$  (straight-line miles)
    # never overestimates the real driving distance, A* still finds the
    # shortest route while expanding fewer cities.
    frontier = [(heuristic(start, goal), 0, [start])]
    visited = set()
    expanded = []

    while frontier:
        f, g, path = heapq.heappop(frontier)
        city = path[-1]
        if city in visited:
            continue
        visited.add(city)
        expanded.append(city)
        if city == goal:
            return path, expanded

        for neighbor, miles in GRAPH[city].items():
            if neighbor not in visited:
                new_g = g + miles
                new_f = new_g + heuristic(neighbor, goal)

```

```

        heapq.heappush(frontier, (new_f, new_g, path + [neighbor]))

    return [], expanded

ALGORITHMS = {"BFS": bfs, "DFS": dfs, "UCS": ucs, "A*": astar}

# -----
# Output: printed summary and travel maps
# -----

def run_all(start, goal):
    h = heuristic(start, goal)
    print(f"\n{start} -> {goal}")
    print(f"Straight-line h = {h:.1f} miles | Airplane @ 250 mph = {h / AIRPLANE_SPEED_MPH:.2f} hours")

    results = {}
    for name, algorithm in ALGORITHMS.items():
        path, expanded = algorithm(start, goal)
        results[name] = (path, expanded)
        print(f" {name:<4} {path_distance(path):>5} miles | {len(expanded):>2} expanded | {' ->'.join(path)}")
    return results

def draw_map(ax, path, expanded, title):
    # Longitude is used as x and latitude as y, so cities appear roughly
    # where they are on a real map.

    # Highways (gray) with driving distances.
    for city in GRAPH:
        for neighbor, miles in GRAPH[city].items():
            if city < neighbor: # each road is stored both ways; draw it once
                lat1, lon1 = CITY_COORDS[city]
                lat2, lon2 = CITY_COORDS[neighbor]
                ax.plot([lon1, lon2], [lat1, lat2], color="lightgray", zorder=1)
                ax.text((lon1 + lon2) / 2, (lat1 + lat2) / 2, str(miles), fontsize=6, color="gray")

    # Final path (blue).
    for i in range(len(path) - 1):
        lat1, lon1 = CITY_COORDS[path[i]]
        lat2, lon2 = CITY_COORDS[path[i + 1]]
        ax.plot([lon1, lon2], [lat1, lat2], color="blue", linewidth=3, zorder=2)

    # Cities: orange = expanded (numbered in expansion order), red = never expanded.
    for city, (lat, lon) in CITY_COORDS.items():
        color = "orange" if city in expanded else "red"
        ax.scatter(lon, lat, color=color, s=60, zorder=3)
        ax.text(lon, lat - 0.6, city, fontsize=7, ha="center")
        if city in expanded:
            ax.text(lon - 0.5, lat + 0.3, str(expanded.index(city) + 1), fontsize=8,
                    color="darkorange", weight="bold")

    ax.set_title(title, fontsize=9)
    ax.set_xticks([])
    ax.set_yticks([])

def show_maps(start, goal, results):
    # One window, 2x2 grid: one map per algorithm for side-by-side comparison.
    fig, axes = plt.subplots(2, 2, figsize=(13, 9))
    fig.suptitle(f"{start} -> {goal} (blue = path, orange # = expansion order)")

    for ax, (name, (path, expanded)) in zip(axes.flat, results.items()):
        title = f"{name}: {path_distance(path)} miles, {len(expanded)} cities expanded"
        draw_map(ax, path, expanded, title)

    plt.tight_layout()
    plt.show()

# -----
# Menu
# -----

```

```

def choose_city(prompt):
    for i, city in enumerate(CITIES, 1):
        print(f" {i:>2}. {city}")
    while True:
        choice = input(prompt)
        if choice.isdigit() and 1 <= int(choice) <= len(CITIES):
            return CITIES[int(choice) - 1]
        print(" Please enter a number from the list.")

def main():
    while True:
        print("\n=== Highway Search ===")
        print("1. Pick a start and target city")
        print("2. Run 5 random tests")
        print("3. Quit")
        option = input("Choose 1, 2, or 3: ")

        if option == "1":
            start = choose_city("Start city number: ")
            goal = choose_city("Target city number: ")
            if start == goal:
                print("Start and target must be different.")
                continue
            results = run_all(start, goal)
            show_maps(start, goal, results)

        elif option == "2":
            # Fixed seed so the same 5 tests appear in the report and the video.
            random.seed(42)
            for test in range(1, 6):
                start, goal = random.sample(CITIES, 2)
                print(f"\n--- Random test {test} ---")
                results = run_all(start, goal)
                show_maps(start, goal, results) # close the window to see the next test

        elif option == "3":
            break

if __name__ == "__main__":
    main()

```

2. Source code: city_data.py

```

"""Data for the 11-city highway search project.

ROAD_EDGES are the team's Google Maps driving distances g(n).
HEURISTIC_PAIRS are the team's straight-line distances h(n), in miles.
"""

AIRPLANE_SPEED_MPH = 250.0

CITIES = [
    "Saint Paul", "Chicago", "Springfield", "Indianapolis", "Columbus",
    "Philadelphia", "New York", "Washington DC", "Columbia", "Atlanta", "Nashville"
]

# Approximate geographic coordinates used ONLY to place nodes on the program's map.
# Search costs use ROAD_EDGES and HEURISTIC_PAIRS below.
CITY_COORDS = {
    "Saint Paul": (44.9537, -93.0900),
    "Chicago": (41.8781, -87.6298),
    "Springfield": (39.7817, -89.6501),
    "Indianapolis": (39.7684, -86.1581),
    "Columbus": (39.9612, -82.9988),
    "Philadelphia": (39.9526, -75.1652),
    "New York": (40.7128, -74.0060),
    "Washington DC": (38.9072, -77.0369),
    "Columbia": (34.0007, -81.0348),
    "Atlanta": (33.7490, -84.3880),
}

```

```

    "Nashville": (36.1627, -86.7816),
}

# g(n): driving-distance edges from the first handwritten data sheet.
ROAD_EDGES = [
    ("Saint Paul", "Chicago", 398),
    ("Saint Paul", "Springfield", 515),
    ("Springfield", "Chicago", 201),
    ("Springfield", "Nashville", 374),
    ("Springfield", "Indianapolis", 211),
    ("Chicago", "Indianapolis", 183),
    ("Chicago", "Columbus", 325),
    ("Indianapolis", "Nashville", 288),
    ("Indianapolis", "Columbus", 174),
    ("Nashville", "Columbia", 462),
    ("Nashville", "Atlanta", 248),
    ("Columbus", "Philadelphia", 469),
    ("Columbus", "Washington DC", 398),
    ("Atlanta", "Columbia", 214),
    ("Columbia", "Washington DC", 478),
    ("Washington DC", "Philadelphia", 136),
    ("Philadelphia", "New York", 95),
]

# h(n): all 55 unique straight-line city-pair distances supplied by the team.
HEURISTIC_PAIRS = {
    ("Saint Paul", "Chicago"): 346.44,
    ("Saint Paul", "Springfield"): 397.21,
    ("Saint Paul", "Indianapolis"): 505.89,
    ("Saint Paul", "Columbus"): 619.46,
    ("Saint Paul", "Philadelphia"): 974.70,
    ("Saint Paul", "New York"): 1007.55,
    ("Saint Paul", "Washington DC"): 922.29,
    ("Saint Paul", "Columbia"): 991.33,
    ("Saint Paul", "Atlanta"): 901.326,
    ("Saint Paul", "Nashville"): 691.88,

    ("Chicago", "Springfield"): 179.51,
    ("Chicago", "Indianapolis"): 165.34,
    ("Chicago", "Columbus"): 291.66,
    ("Chicago", "Philadelphia"): 663.84,
    ("Chicago", "New York"): 711.148,
    ("Chicago", "Washington DC"): 594.225,
    ("Chicago", "Columbia"): 652.39,
    ("Chicago", "Atlanta"): 588.091,
    ("Chicago", "Nashville"): 397.628,

    ("Springfield", "Indianapolis"): 186.0,
    ("Springfield", "Columbus"): 352.246,
    ("Springfield", "Philadelphia"): 767.487,
    ("Springfield", "New York"): 824.93,
    ("Springfield", "Washington DC"): 675.288,
    ("Springfield", "Columbia"): 621.64,
    ("Springfield", "Atlanta"): 508.823,
    ("Springfield", "Nashville"): 296.54,

    ("Indianapolis", "Columbus"): 169.24,
    ("Indianapolis", "Philadelphia"): 583.29,
    ("Indianapolis", "New York"): 643.35,
    ("Indianapolis", "Washington DC"): 492.189,
    ("Indianapolis", "Columbia"): 488.603,
    ("Indianapolis", "Atlanta"): 426.476,
    ("Indianapolis", "Nashville"): 251.50,

    ("Columbus", "Philadelphia"): 414.98,
    ("Columbus", "New York"): 475.44,
    ("Columbus", "Washington DC"): 326.60,
    ("Columbus", "Columbia"): 426.119,
    ("Columbus", "Atlanta"): 435.507,
    ("Columbus", "Nashville"): 333.711,

    ("Philadelphia", "New York"): 80.62,
    ("Philadelphia", "Washington DC"): 123.41,
    ("Philadelphia", "Columbia"): 523.569,
    ("Philadelphia", "Atlanta"): 663.913,
    ("Philadelphia", "Nashville"): 683.559,
}

```

```

("New York", "Washington DC"): 202.26,
("New York", "Columbia"): 523.73,
("New York", "Atlanta"): 745.027,
("New York", "Nashville"): 682.955,

("Washington DC", "Columbia"): 405.32,
("Washington DC", "Atlanta"): 542.443,
("Washington DC", "Nashville"): 566.509,

("Columbia", "Atlanta"): 193.14,
("Columbia", "Nashville"): 357.54,

("Atlanta", "Nashville"): 214.81,
}

GRAPH = {city: {} for city in CITIES}
for a, b, miles in ROAD_EDGES:
    GRAPH[a][b] = miles
    GRAPH[b][a] = miles

def heuristic(city, goal):
    """Return h(city, goal) in straight-line miles."""
    if city == goal:
        return 0.0
    return HEURISTIC_PAIRS.get((city, goal), HEURISTIC_PAIRS.get((goal, city)))

```